

1. exception handling ->
2. stack and queue
3. garbage collection
4. args and kwargs -> done
5. numpy arrays -> done
6. identity operators -> done

Identity operators

```
2 == 2
```

True

```
2 is 2
```

```
<>:1: SyntaxWarning: "is" with 'int' literal. Did you mean "=="?
```

```
<>:1: SyntaxWarning: "is" with 'int' literal. Did you mean "=="?
```

```
C:\Users\s_nas\AppData\Local\Temp\ipykernel_6076\564785565.py:1:
```

```
SyntaxWarning: "is" with 'int' literal. Did you mean "=="?
```

```
2 is 2
```

True

```
2 != 2
```

False

```
2 is not 2
```

```
<>:1: SyntaxWarning: "is not" with 'int' literal. Did you mean "!="?
```

```
<>:1: SyntaxWarning: "is not" with 'int' literal. Did you mean "!="?
```

```
C:\Users\s_nas\AppData\Local\Temp\ipykernel_6076\3997639733.py:1:
```

```
SyntaxWarning: "is not" with 'int' literal. Did you mean "!="?
```

```
2 is not 2
```

False

```
2 is not '2'
```

```
<>:1: SyntaxWarning: "is not" with 'int' literal. Did you mean "!="?
```

```
<>:1: SyntaxWarning: "is not" with 'int' literal. Did you mean "!="?
```

```
C:\Users\s_nas\AppData\Local\Temp\ipykernel_6076\3071503370.py:1:
```

```
SyntaxWarning: "is not" with 'int' literal. Did you mean "!="?
```

```
2 is not '2'
```

True

```
2 is '2'
```

```
<>:1: SyntaxWarning: "is" with 'int' literal. Did you mean "=="?
```

```
<>:1: SyntaxWarning: "is" with 'int' literal. Did you mean "=="?
```

```
C:\Users\s_nas\AppData\Local\Temp\ipykernel_6076\1162160561.py:1:
```

```
SyntaxWarning: "is" with 'int' literal. Did you mean "=="?  
2 is '2'
```

```
False
```

```
a = 2  
b = 2
```

```
a == b
```

```
True
```

```
a is b
```

```
True
```

```
a is not b
```

```
False
```

```
print('my name is {}'.format('nashit'))
```

```
my name is nashit
```

```
name = 'Nashit'  
print(f'my name is {name}')
```

```
my name is Nashit
```

```
print('i love {}, {}, {}'.format('HTML', 'CSS', 'jQuery', 'Java'))
```

```
i love HTML, CSS, JQuery
```

```
print('i love {1}, {3}, {0}'.format('HTML', 'CSS', 'jQuery', 'Java'))
```

```
i love CSS, Java, HTML
```

```
kwargs = {'name' : 'Nashit', 'city' : 'Patna', 'sub': 'Python'}
```

```
print('hello {name}. you love {sub}'.format(**kwargs))
```

```
hello Nashit. you love Python.
```

```
def a(args):  
    print('hello', args)
```

```
a('nashit')
```

```
hello nashit
```

```
def b(n1,n2,n3):  
    print('hello', n1)
```

```

def c(*args):
    for i in args:
        print('hello', i)

c('Nashit', 'Rahul', 'Amit')

hello Nashit
hello Rahul
hello Amit

def c(*a):
    for i in a:
        print('hello', i)

c('Nashit', 'Rahul', 'Amit')

hello Nashit
hello Rahul
hello Amit

def d(**kwargs):
    print('Congratulations', kwargs['First'])

d(First='nashit', Second = 'Rahul', Third = 'Amit')

Congratulations nashit

def e(*args):
    print('hello', args[1])

e('Nashit', 'Rahul', 'Ranjit')

hello Rahul

def f(x):
    print(x)

f(7, 4)

```

```

-----
-----
TypeError                                Traceback (most recent call
last)
Cell In[37], line 1
----> 1 f(7, 4)

```

TypeError: f() takes 1 positional argument but 2 were given

```

def f(*x):
    print(x[0])

f(7, 4)

```

7

```
a = 5
type(a, a)
```

```
-----
-----
```

```
TypeError                                Traceback (most recent call
last)
```

```
Cell In[41], line 2
```

```
      1 a = 5
----> 2 type(a, a)
```

```
TypeError: type() takes 1 or 3 arguments
```

```
def g(a):
    print(type(a))
```

```
g(5)
```

```
<class 'int'>
```

```
def h(*a):
    print(type(a[0]))
```

```
h(4, 6)
```

```
<class 'int'>
```

```
exception handling
```